



Licenciatura en Ciencias Computacionales

7° 2

3.2 PRACTICA FRAGMENTOS

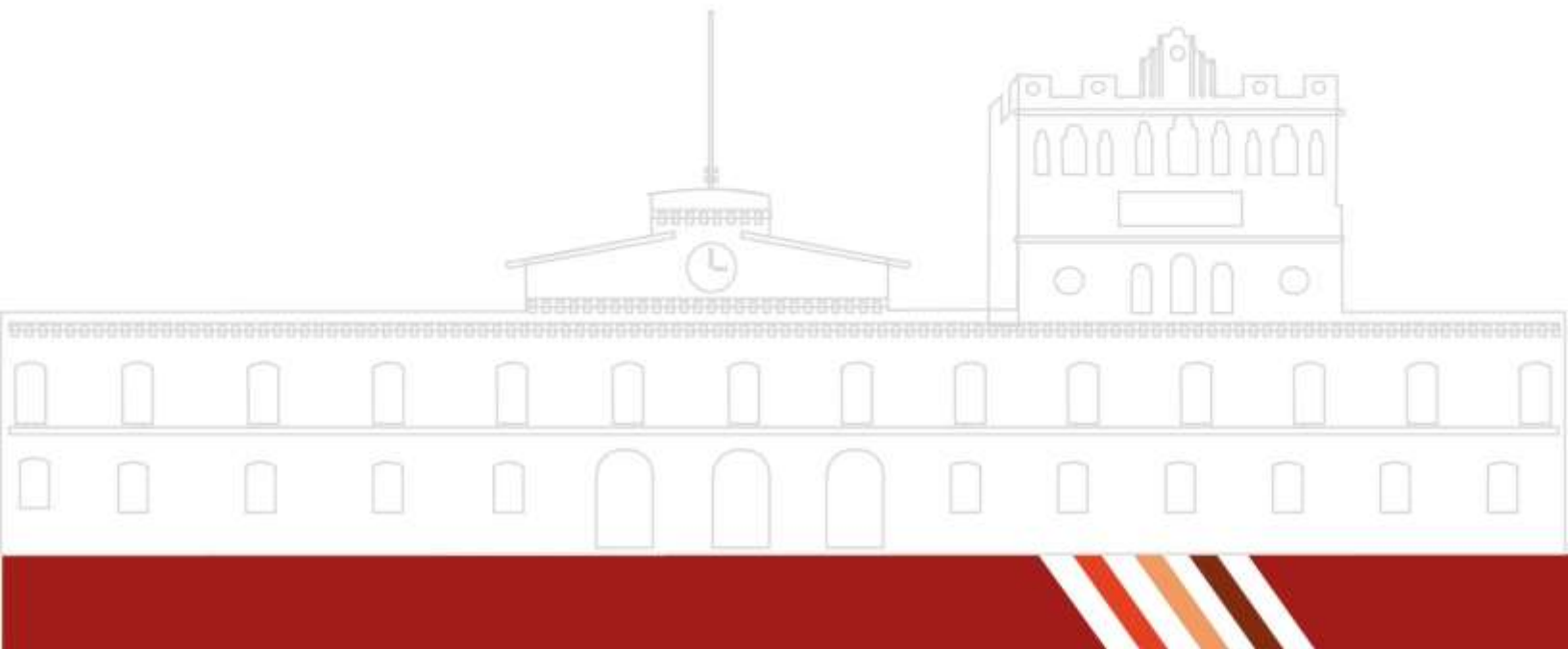
BASES DE DATOS DISTRIBUIDAS

Catedrático:

DR. EDUARDO CORNEJO VELAZQUEZ

Presenta:

VARGAS FUENTES REY



INTRODUCCIÓN

La administración de una flotilla vehicular exige controlar aspectos como el mantenimiento, la documentación, los conductores y las rutas. Para ello se diseñó una **base de datos relacional** que organiza estas entidades y asegura la integridad de la información mediante llaves primarias y foráneas. Con este modelo es posible consultar de forma eficiente el historial de cada vehículo, la vigencia de sus documentos y las asignaciones de conductores y rutas, optimizando así la operación de la flotilla.

MARCO TEORICO

Procedimientos almacenados (Stored Procedures)

Un procedimiento almacenado consiste en un conjunto de instrucciones SQL o PL/SQL que se guardan directamente dentro del servidor de base de datos y pueden ejecutarse posteriormente con una simple llamada. Su finalidad es automatizar operaciones frecuentes o complejas, además de concentrar la lógica del sistema en un solo punto.

Estos procedimientos pueden recibir parámetros, ejecutar varias consultas o actualizaciones dentro de una misma rutina y ser invocados por otras funciones o aplicaciones, lo cual facilita la reutilización del código y mejora la eficiencia en el manejo de datos.

Funciones (Functions)

Las funciones comparten muchas características con los procedimientos, pero se diferencian porque devuelven obligatoriamente un valor al concluir su ejecución. Son útiles para realizar operaciones aritméticas, validar información o generar resultados específicos que pueden utilizarse dentro de otras consultas SQL o en diferentes partes del sistema.

Asimismo, las funciones permiten incluir parámetros de entrada y emplear estructuras de control que les otorgan flexibilidad para realizar tareas más elaboradas o personalizadas.

Estructuras de control condicionales y repetitivas

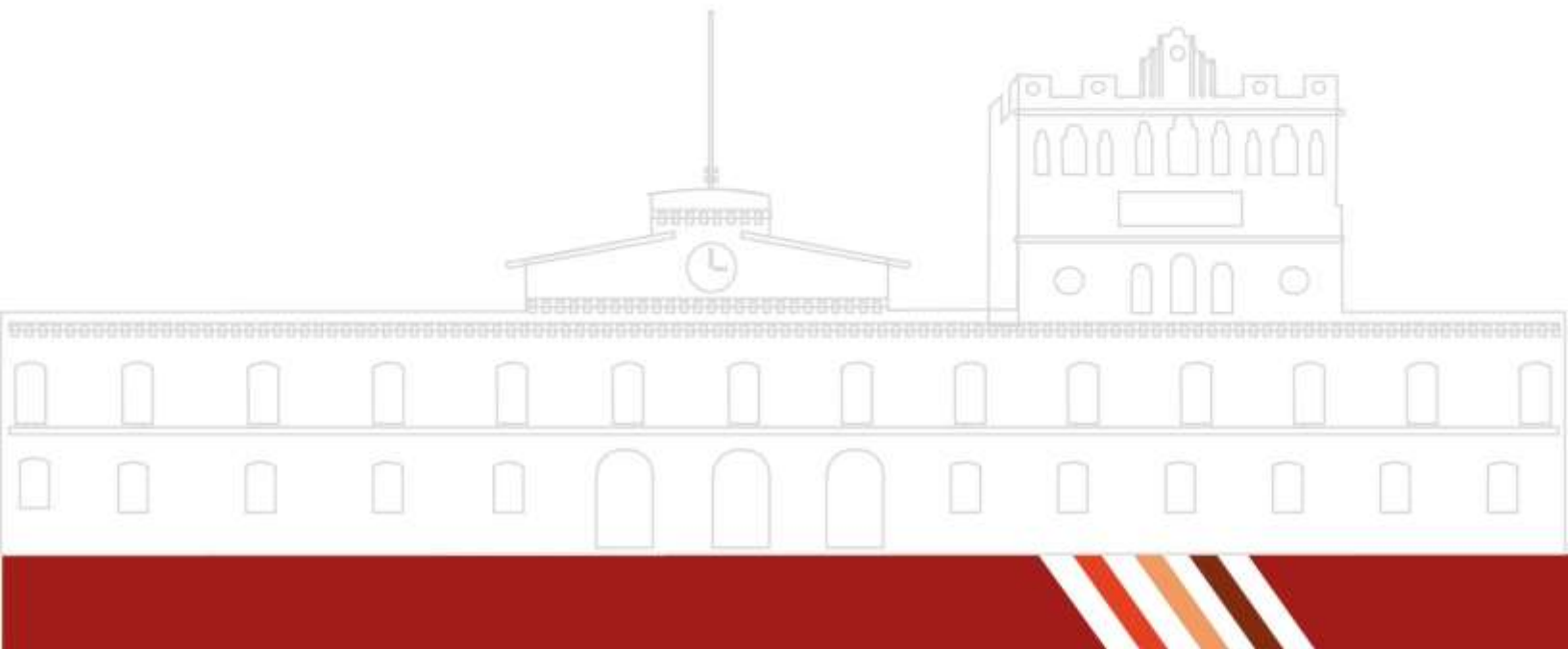
Las estructuras condicionales sirven para tomar decisiones en el flujo del programa, ejecutando diferentes acciones según el cumplimiento de una condición lógica, como ocurre con las sentencias IF o CASE.

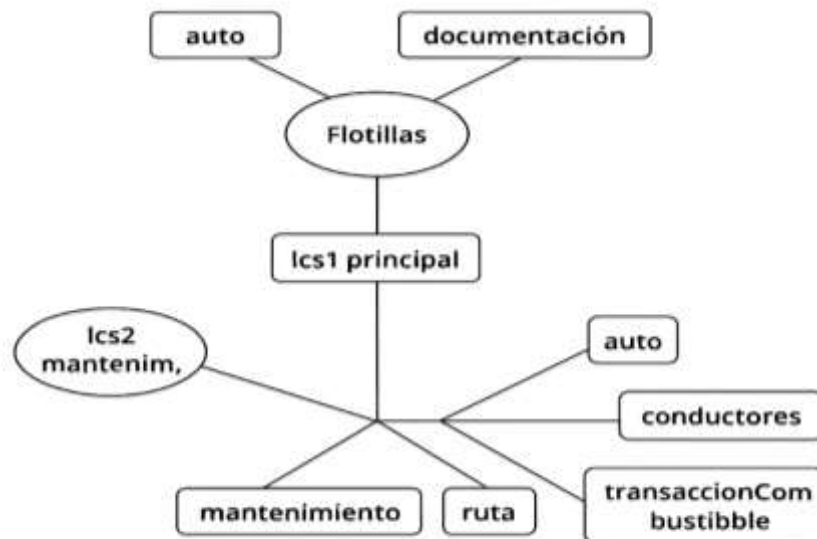
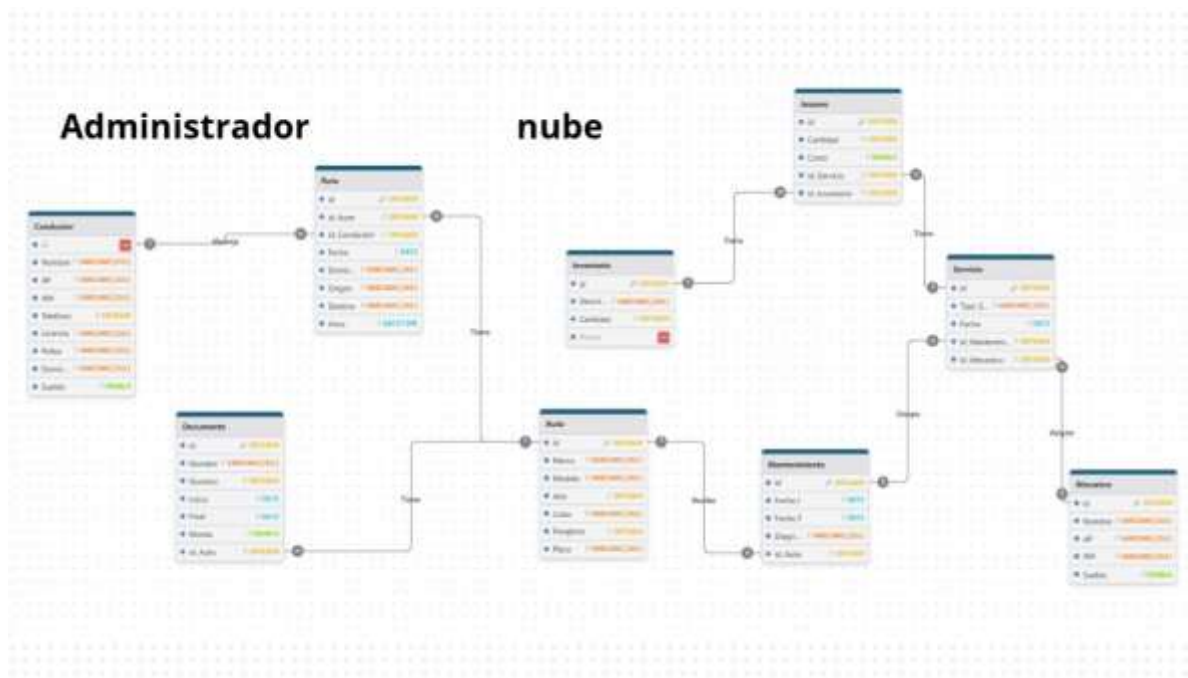
Por otra parte, las estructuras repetitivas permiten que un bloque de código se ejecute varias veces, ya sea un número definido de ocasiones o mientras se mantenga cierta condición. Ejemplos de ellas son FOR, WHILE y LOOP. Estas herramientas resultan esenciales para automatizar procesos y manipular conjuntos de datos de manera controlada dentro de los procedimientos y funciones.

Disparadores (Triggers)

Los disparadores son fragmentos de código que se ejecutan de forma automática cuando ocurre un evento determinado en la base de datos, como una inserción, actualización o eliminación de registros. Su propósito es mantener la integridad y coherencia de los datos, además de reducir la intervención manual en tareas rutinarias.

Pueden configurarse para activarse antes o después del evento que los desencadena y suelen emplearse para auditorías, validaciones o actualizaciones automáticas. Gracias a su ejecución implícita, los triggers aseguran que las reglas del sistema se cumplan sin depender de la acción directa del usuario.





Script de creación de nodos

lcs1 principal

```
CREATE DATABASE lcs1 principal;
```

```
USE lcs1 principal;
```

```
CREATE TABLE auto (
```

```
id INT NOT NULL,
```

```
marca VARCHAR(255),
```

```
modelo VARCHAR(255),
```

```
anio INT,
```

```
color VARCHAR(255),
```

```
pasajeros INT,
```

```
placa VARCHAR(255),
```

```
PRIMARY KEY(id)
```

```
);
```

```
CREATE TABLE documentacion (
```

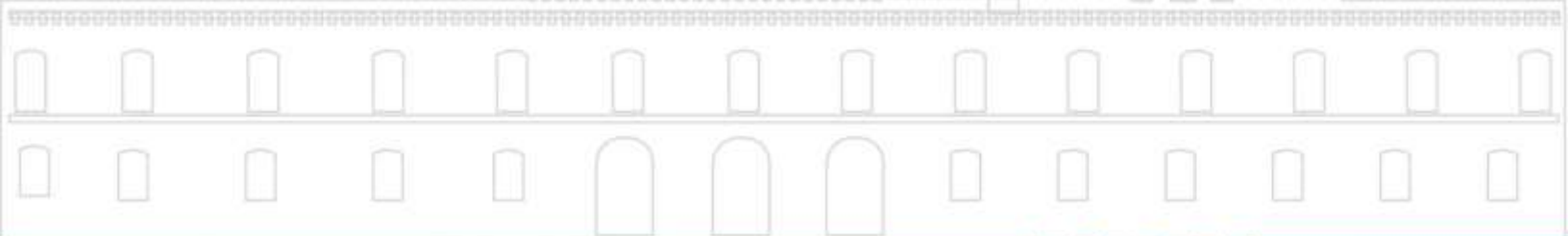
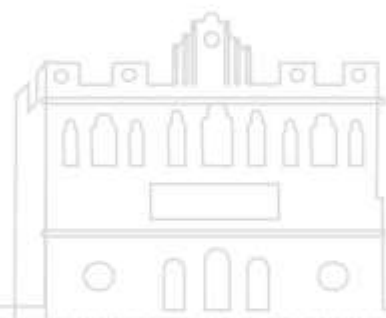
```
id INT NOT NULL,
```

```
nombre VARCHAR(255),
```

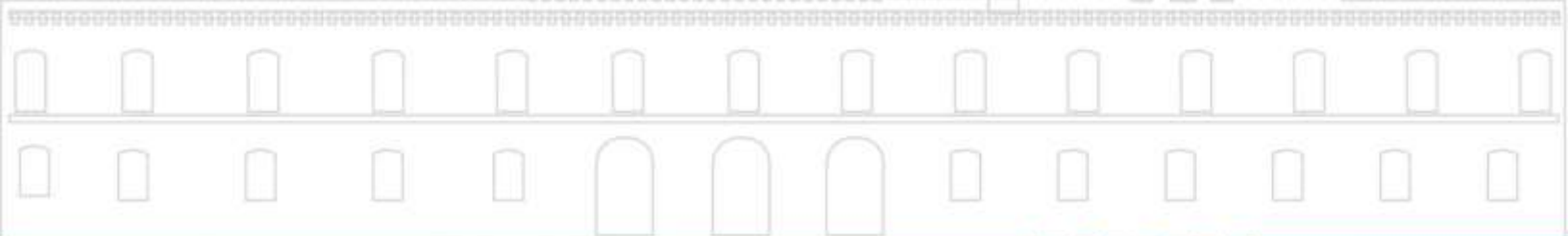
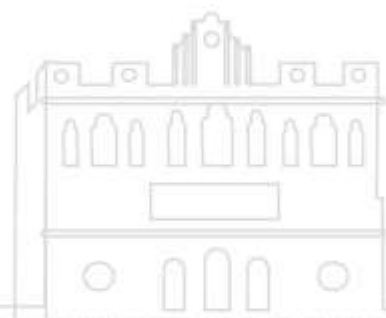
```
numero INT,
```

```
inicio DATE,
```

```
final DATE,
```



```
monto NUMERIC,  
idAuto INT,  
PRIMARY KEY(id),  
FOREIGN KEY (idAuto) REFERENCES auto(id)  
);  
lcs2 mantenimiento  
CREATE DATABASE lcs2 mantenimiento;  
USE lcs2 mantenimiento;  
CREATE TABLE auto (  
id INT NOT NULL,  
marca VARCHAR(255),  
modelo VARCHAR(255),  
anio INT,  
color VARCHAR(255),  
pasajeros INT,  
placa VARCHAR(255),  
PRIMARY KEY(id)  
);  
CREATE TABLE mantenimiento (  
id INT NOT NULL,  
fechaInicio DATE,  
fechaFinal DATE,
```



diagnostico VARCHAR(255),
descripcion VARCHAR(255),
idAuto INT,
PRIMARY KEY(id),
FOREIGN KEY (idAuto) REFERENCES auto(id)
);

lcs3 rutas

CREATE DATABASE lcs3 rutas;

USE lcs3 rutas;

CREATE TABLE auto (

id INT NOT NULL,

marca VARCHAR(255),

modelo VARCHAR(255),

anio INT,

color VARCHAR(255),

pasajeros INT,

placa VARCHAR(255),

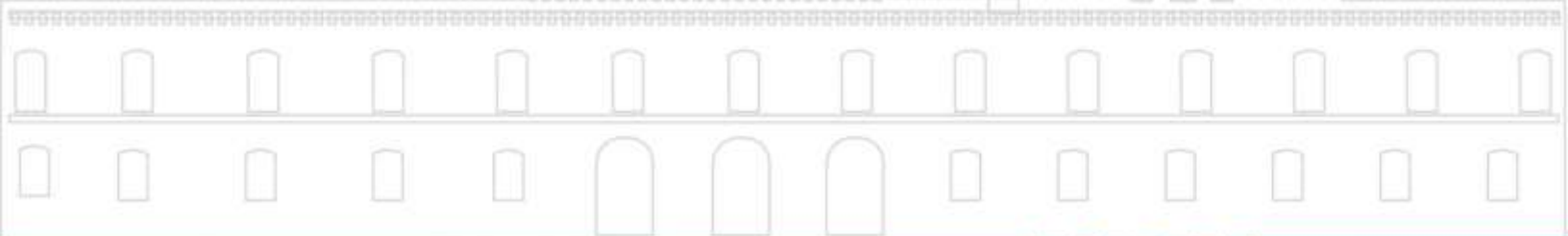
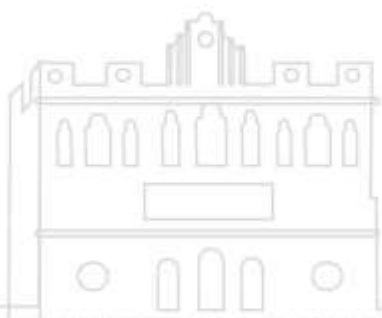
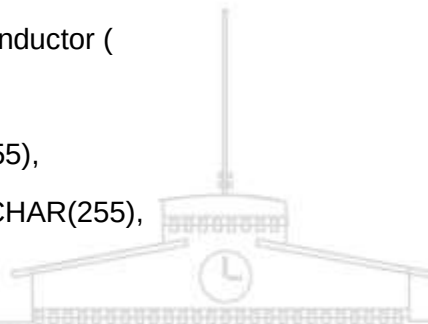
PRIMARY KEY(id)

); CREATE TABLE conductor (

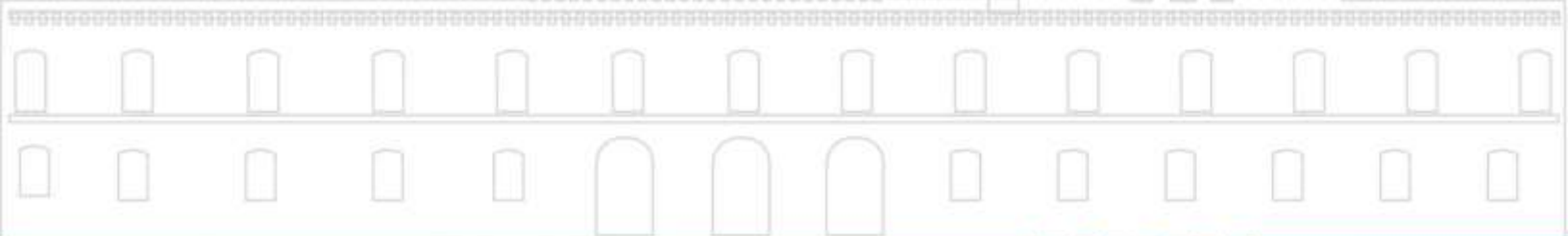
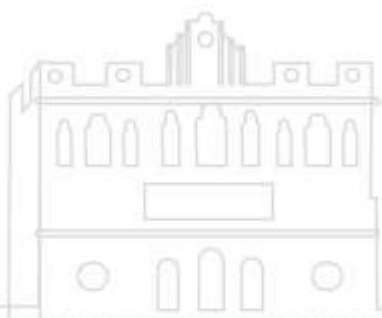
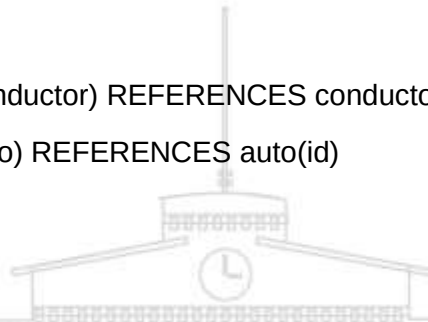
id INT NOT NULL,

nombre VARCHAR(255),

apellidoPaterno VARCHAR(255),



```
apellidoMaterno VARCHAR(255),
telefono VARCHAR(255),
licencia VARCHAR(255),
poliza VARCHAR(255),
domicilio VARCHAR(255),
sueldo DECIMAL,
PRIMARY KEY(id)
);
CREATE TABLE ruta (
id INT NOT NULL,
fecha DATE,
horaInicio TIME,
origen VARCHAR(255),
destino VARCHAR(255),
horaFinal TIME,
cobro DECIMAL,
idConductor INT,
idAuto INT,
PRIMARY KEY(id),
FOREIGN KEY (idConductor) REFERENCES conductor(id),
FOREIGN KEY (idAuto) REFERENCES auto(id)
);
```




```
CREATE TABLE transaccionCombustible (  
id INT NOT NULL,  
idAuto INT,  
fecha DATE,  
cantidadLitros DECIMAL(10, 2),  
costo DECIMAL(10, 2),  
proveedor VARCHAR(255),  
PRIMARY KEY(id),  
FOREIGN KEY (idAuto) REFERENCES auto(id)  
);
```

Scripts de extracción de datos

lcs1 principal

```
SELECT * FROM auto;
```

```
SELECT * FROM documentacion;
```

lcs2 mantenimiento

```
SELECT * FROM auto;
```

```
SELECT * FROM mantenimiento;
```

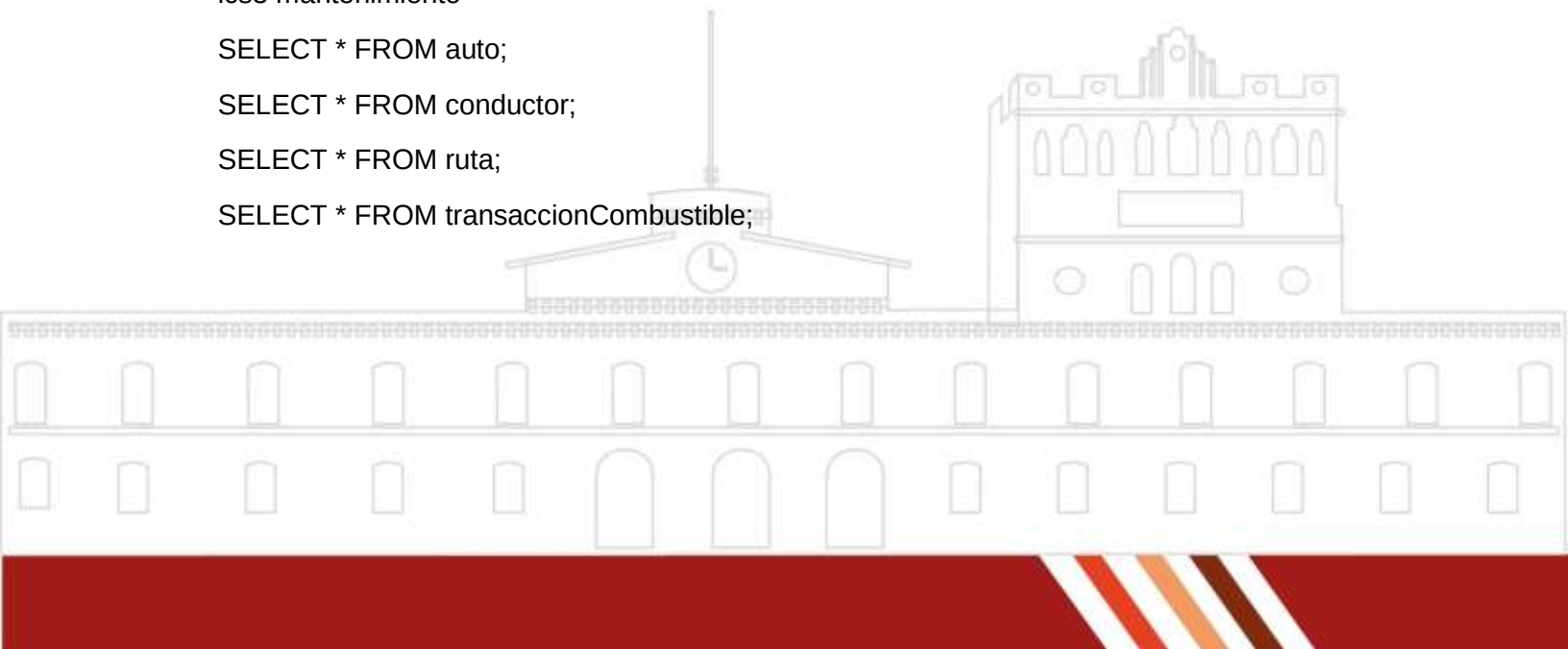
lcs3 mantenimiento

```
SELECT * FROM auto;
```

```
SELECT * FROM conductor;
```

```
SELECT * FROM ruta;
```

```
SELECT * FROM transaccionCombustible;
```



Script de carga de datos

lcs1 principal

```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs1 principal\auto.csv'
```

```
ignore into table lcs1 principal.auto
```

```
fields terminated by ','
```

```
enclosed by ''
```

```
lines terminated by "
```

```
ignore 1 lines
```

```
(id, marca, modelo, anio, color, pasajeros, placa);
```

```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs1 principal\documentacion.csv'
```

```
ignore into table lcs1 principal.documentacion
```

```
fields terminated by ','
```

```
enclosed by ''
```

```
lines terminated by "
```

```
ignore 1 lines
```

```
(id, nombre, numero, inicio, final, monto, idAuto);
```

lcs3 mantenimiento

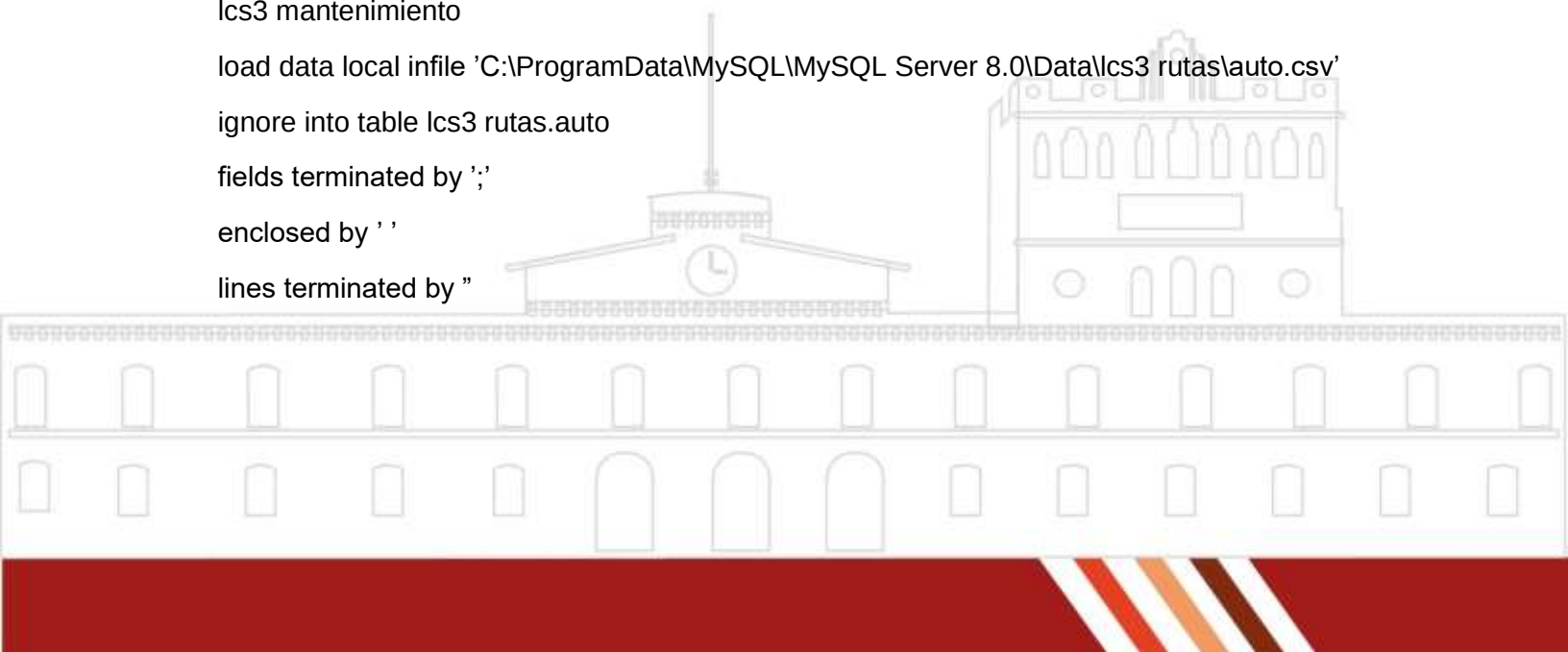
```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs3 rutas\auto.csv'
```

```
ignore into table lcs3 rutas.auto
```

```
fields terminated by ','
```

```
enclosed by ''
```

```
lines terminated by "
```





ignore 1 lines

(id, marca, modelo, anio, color, pasajeros, placa);

```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs3
rutas\conductor.csv'
```

ignore into table lcs3 rutas.conductor

fields terminated by ','

enclosed by ''

lines terminated by "

ignore 1 lines

(id, nombre, apellidoPaterno, apellidoMaterno, telefono, licencia, poliza, domicilio, sueldo);

```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs3 rutas\ruta.csv'
```

ignore into table lcs3 rutas.ruta

fields terminated by ','

enclosed by ''

lines terminated by "

ignore 1 lines

(id, fecha, horaInicio, origen, destino, horaFinal, cobro, idConductor, idAuto);

```
load data local infile 'C:\ProgramData\MySQL\MySQL Server 8.0\Data\lcs3
rutas\transaccionCombustible
```

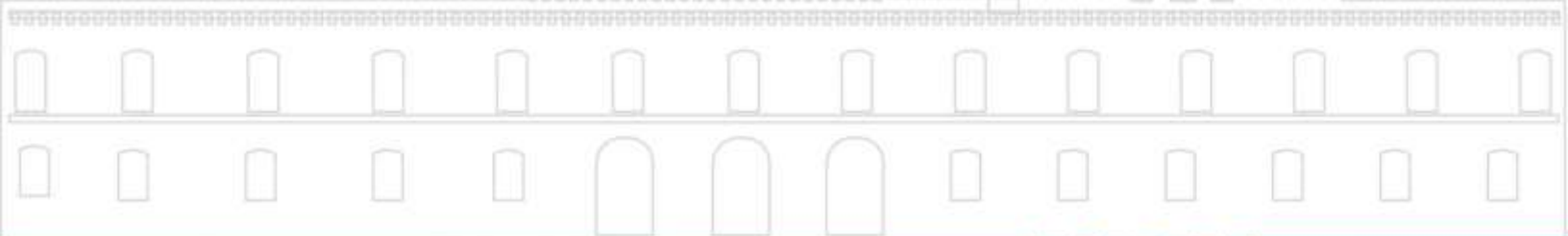
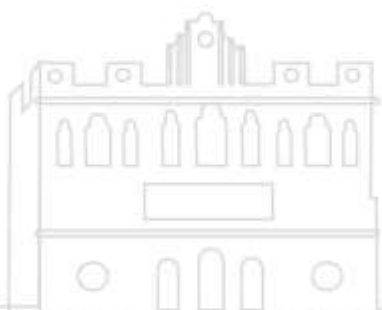
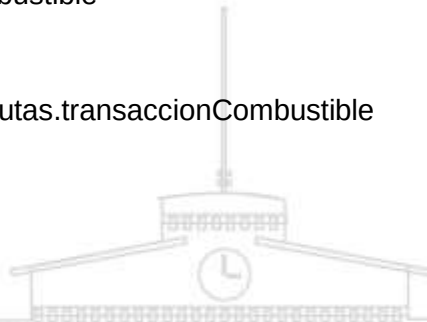
.csv'

ignore into table lcs3 rutas.transaccionCombustible

fields terminated by ','

enclosed by ''

lines terminated by "





ignore 1 lines

(id, idAuto, fecha, cantidadLitros, costo, proveedor);

Script de consulta de datos

lcs1 principal

USE lcs1 principal;

SELECT

a.id,

a.marca,

a.modelo,

a.anio,

d.nombre AS tipo documento,

d.numero,

d.inicio,

d.final

FROM auto a

LEFT JOIN documentacion d ON a.id = d.idAuto;

lcs3 mantenimiento

USE lcs3 rutas;

SELECT

r.id,

r.fecha,

r.horaInicio,

r.origen,



```
r.destino,  
r.horaFinal,  
r.cobro,  
c.nombre AS nombre conductor,  
c.apellidoPaterno,  
a.marca,  
a.modelo,  
a.placa  
FROM ruta r  
INNER JOIN conductor c ON r.idConductor = c.id  
INNER JOIN auto a ON r.idAuto = a.id;
```

CONCLUSION

El conjunto de scripts desarrollados permite la creación, carga y consulta de bases de datos distribuidas orientadas a la gestión de flotillas de vehículos. A través de la definición de tres nodos principales lcs1 principal, lcs2 mantenimiento y lcs3 rutas se logra una estructura modular que separa las funciones administrativas, de mantenimiento y de control operativo, facilitando la organización y el manejo de la información.

Cada nodo mantiene su propio conjunto de tablas relacionadas mediante claves foráneas, garantizando la integridad referencial y permitiendo una fácil integración de datos. El uso de comandos LOAD DATA optimiza el proceso de carga masiva desde archivos externos, mientras que las consultas SQL demuestran la capacidad del sistema para vincular información entre entidades clave, como vehículos, conductores, rutas y documentación.



En conclusión, este modelo de bases de datos distribuidas ofrece una solución eficiente y escalable para la administración integral de flotillas, mejorando la trazabilidad de los vehículos, el control de mantenimiento y la gestión operativa de rutas y recursos humanos.

